Cross-Origin Data Exfiltration via Misconfigured CORS Policy

Severity: **Critical**

CVSS Score: **9.2 (Critical)**

CWE

- **CWE-942:** Permissive Cross-domain Policy with Untrusted Domains
- **CWE-346:** Origin Validation Error
- **CWE-352:** Cross-Site Request Forgery (CSRF) – related impact

Affected Asset

- **URL**: https://crm-stage.crusource.com
- **Parameter/Storage Key**: 'Origin' HTTP Request Header
- **Environment**: Staging
- **Date Discovered**: 11-10-2026

Why this is a vulnerability:

- The CRM application implements Cross-Origin Resource Sharing (CORS) in a way that trusts **any origin supplied by the client**. When a request includes an Origin header, the server reflects that value back in the Access-Control-Allow-Origin response header and simultaneously sets Access-Control-Allow-Credentials: true
- This combination is dangerous. It instructs the browser that: The requesting origin is explicitly trusted, and credentials (cookies, session tokens, 'authorization' headers) may be included in the cross-origin request.
- Because the server does not validate the Origin against an allowlist, an attacker can host a malicious page that silently makes authenticated requests to the CRM API and reads the responses. The victim's browser will attach their valid session cookie and the server will authorize the request – but the response is delivered to the attacker's JavaScript.

Delivery vector (how an attacker would steal it):

An attacker who can lure a logged-in CRM user to a malicious website can:

- **Read any CRM data the victim can access** — customer PII, contacts, deals, tickets, internal notes, and account settings.
- **Perform state-changing actions** on behalf of the victim via cross-origin fetch/XHR (create, update, or delete records, modify users, change settings).
- **Exfiltrate the victim's data** to an attacker-controlled server in real time.
- **Escalate privilege** if the victim is an administrator, potentially leading to full tenant compromise.
- **Bypass authentication entirely —** the victim's existing session cookie is sufficient; no credentials are needed by the attacker.

Remediation recommendation:

1. Implement a strict server-side allowlist of trusted origins. Compare the Origin header against this list using an exact-match check.
2. Never reflect the Origin header dynamically without allowlist validation.
3. Avoid wildcard * with credentials — browsers block this combination, but its presence indicates a misconfiguration intent.
4. Do not trust null origins in production. Reject them unless explicitly required.
5. Restrict Access-Control-Allow-Methods and Access-Control-Allow-Headers to only those required by the application.
6. Use CSRF tokens as defense-in-depth for state-changing endpoints.
7. Set Vary: Origin on responses to prevent caching of CORS headers across different origins.

Summary

The CRM application at https://crm-stage.crusource.com reflects arbitrary client-supplied Origin headers in its Access-Control-Allow-Origin response while setting Access-Control-Allow-Credentials: true. This critical CORS misconfiguration allows any attacker-controlled website to make authenticated cross-origin requests on behalf of a logged-in CRM user and read the responses. An attacker can silently exfiltrate sensitive customer data, modify records, and potentially escalate to full administrative takeover. The server must validate the Origin header against a strict allowlist of trusted domains and must never reflect unvalidated origins when credentials are permitted.